

FactoryGuard AI

Intelligent Manufacturing Quality Decision System

Detailed Project & Concept Report

(Structured for Project Documentation and Interview Preparation)

GitHub Repository:

<https://github.com/Bhardwaj-Arin/FactoryGuard-AI-Intelligent-Manufacturing-Quality-Decision-System>

Live Streamlit App: <https://factoryguard-ai-intelligent-manufacturing-quality-decision-sys.streamlit.app/>

Domain: Manufacturing & Industrial Automation | Problem Type: Supervised Binary Classification | Final Model: Random Forest Classifier

This report explains not only *what* was built, but *why* every major decision was made - the concept behind each technique, the need it addressed, and the outcome it produced - and closes with a dedicated interview-preparation Q&A section covering the questions most likely to come up about this project.

Table of Contents

1. Executive Summary
2. Business Understanding
 - 2.1 Business Problem
 - 2.2 Objectives & Stakeholders
 - 2.3 Why Machine Learning
3. Data Understanding
 - 3.1 Dataset Description
 - 3.2 Data Quality
 - 3.3 Target Variable & Class Imbalance
4. Exploratory Data Analysis (EDA)
 - 4.1 What is EDA and Why It Matters
 - 4.2 Univariate & Bivariate Analysis
 - 4.3 Correlation Analysis
 - 4.4 Outliers & Skewness
 - 4.5 Key EDA Findings
5. Feature Engineering & Preprocessing
 - 5.1 Target Leakage
 - 5.2 Train-Test Split
 - 5.3 Derived Features
 - 5.4 Encoding & Scaling
 - 5.5 The Preprocessing Pipeline
6. Model Development
 - 6.1 Framing the ML Problem
 - 6.2 Candidate Algorithms Explained
 - 6.3 Baseline Comparison
 - 6.4 Cross-Validation
 - 6.5 Hyperparameter Tuning
 - 6.6 Handling Class Imbalance
7. Model Evaluation & Metrics
 - 7.1 Confusion Matrix
 - 7.2 Precision, Recall, F1-score
 - 7.3 ROC-AUC vs. PR-AUC
 - 7.4 Final Model Selection
 - 7.5 Decision Threshold Tuning
8. Model Interpretation & Explainability
 - 8.1 Feature Importance
 - 8.2 Business Interpretation
9. Deployment & the Streamlit Application
 - 9.1 Model Persistence
 - 9.2 Application Architecture

9.3 Application Pages

10. Technology Stack

11. Limitations & Future Improvements

12. Conclusion

13. Interview Preparation - Q&A

Project Overview Questions

Data & EDA Questions

Feature Engineering Questions

Model & Algorithm Questions

Evaluation Metric Questions

Deployment Questions

General / Conceptual ML Questions

1. Executive Summary

FactoryGuard AI is an end-to-end machine learning decision-support system that predicts whether a manufactured unit is likely to pass final quality control or is potentially problematic, based on structured manufacturing and process data. It was built as a complete applied ML project: business framing, data understanding, exploratory data analysis, feature engineering, comparative model development, evaluation, interpretation, and deployment as a live, interactive web application.

This report is written at two levels simultaneously. First, as a normal project report - describing what was done at each stage. Second, as an interview-preparation resource - explaining *why* each technique was chosen, what concept it represents, what alternative existed, and what the actual outcome was, so that every decision in the project can be defended and explained clearly in conversation.

Item	Detail
Problem Type	Supervised binary classification
Final Model	Random Forest Classifier (tuned via RandomizedSearchCV)
Target Variable	final_pass (1 = passed QC, 0 = potentially problematic)
Dataset Size	10,000 records, 23 raw features, 115 features after encoding
Class Balance	87.61% majority (pass) / 12.39% minority (problematic)
Train / Test Split	8,000 / 2,000 records (80/20, stratified)
Test Accuracy	85.35%
Test ROC-AUC	70.09%
Decision Threshold	0.50 (kept after comparison with 0.55)
Deployment	Streamlit multipage web app (5 pages), backed by a shared src/ pipeline

Positioning: *FactoryGuard AI is explicitly designed and documented as a screening and decision-support tool, not an autonomous quality-control gate. Every prediction is a signal for human review, not an automatic accept/reject decision - this framing matters because it shapes how every metric in this report should be interpreted.*

2. Business Understanding

2.1 Business Problem

Manufacturing organizations generate enormous volumes of operational data - production volume, cost, machine utilization, and quality scores - yet quality-related decisions are often still made reactively, through manual review of historical reports rather than systematic analysis. As production complexity grows, spotting meaningful patterns manually becomes harder, and quality issues tend to be discovered *after* they occur rather than anticipated in advance.

Core problem statement: How can manufacturing companies leverage historical production and operational data to make smarter quality-related decisions and optimize overall production efficiency through data-driven intelligence?

2.2 Objectives & Stakeholders

- Improve manufacturing quality through data-driven decision making
- Support production managers with fast, consistent quality signals
- Shift from reactive to proactive quality management
- Reduce unnecessary inspection effort while still catching real problems
- Keep the system interpretable and auditable rather than a black box

Stakeholder	How They Benefit
Production Managers	Faster, more consistent quality-related decisions
Quality Engineers	Data-backed prioritization of which units to inspect further
Machine Operators	Visibility into which process conditions drive outcomes
Business Executives	Manufacturing KPIs grounded in predictive analytics

2.3 Why Machine Learning (and not just rules or manual review)

Rule-based / threshold-based monitoring (e.g. "flag if temperature > X") is simple but only catches problems along the one dimension it checks. Real manufacturing defects usually emerge from the *interaction* of several variables at once (e.g. an aging machine, running fast, on a humid day, with a lower material grade) - a pattern no single fixed rule can express. Machine learning models like Random Forest can learn these multi-variable, nonlinear interactions directly from historical outcomes, without a human having to hand-write every combination of rules.

Important framing: the project deliberately treats the model as a decision-support system. Human expertise remains essential for validating recommendations, interpreting business context, and making the final call - the model's job is to surface signal from noise faster than manual review could.

3. Data Understanding

3.1 Dataset Description

The Manufacturing Quality Decisions dataset contains 10,000 manufacturing event records. Each row represents a single manufacturing event and captures production, machine, material, environmental, inspection, and cost-related information for that event.

Attribute	Value
Records	10,000
Raw Features	23
Categorical Features	8 - plant, line, shift, machine_id, operator_id, material_grade, inspection_method, defect_type
Numerical Features	13 - machine age, temperature, humidity, process speed, cycle time, energy, cost, etc.
Datetime Feature	1 - event_ts
Identifier Feature	1 - event_id (excluded from modeling; used only for traceability)
Missing Values	None
Duplicate Records	None

3.2 Data Quality

Before any modeling, the dataset was checked for missing values, duplicate rows, and consistent data types. A dataset with no missing values or duplicates is a strong starting point - it means effort can go straight into feature engineering rather than data cleaning, but it also means results are attributable to real relationships in the data rather than artifacts of cleaning decisions.

3.3 Target Variable & Class Imbalance

The target variable is **final_pass**: 1 if the unit passed final quality inspection, 0 if it was flagged as potentially problematic. The dataset also contains related outcome fields (scrap, warranty_claim_90d) that describe what happened *after* the quality decision - these are discussed in Section 5.1 on target leakage.

What is class imbalance, and why does it matter here? Class imbalance means one class dominates the dataset - here, 87.6% of units passed and only 12.4% were problematic. This matters because a naive model can achieve ~87.6% accuracy simply by predicting "pass" for every single record, without learning anything useful about the minority class. Since the whole point of FactoryGuard AI is to catch the rare problematic cases, accuracy alone is a misleading metric here, and both the evaluation strategy (Section 7) and the modeling strategy (Section 6.6) had to be built around this imbalance from the start.

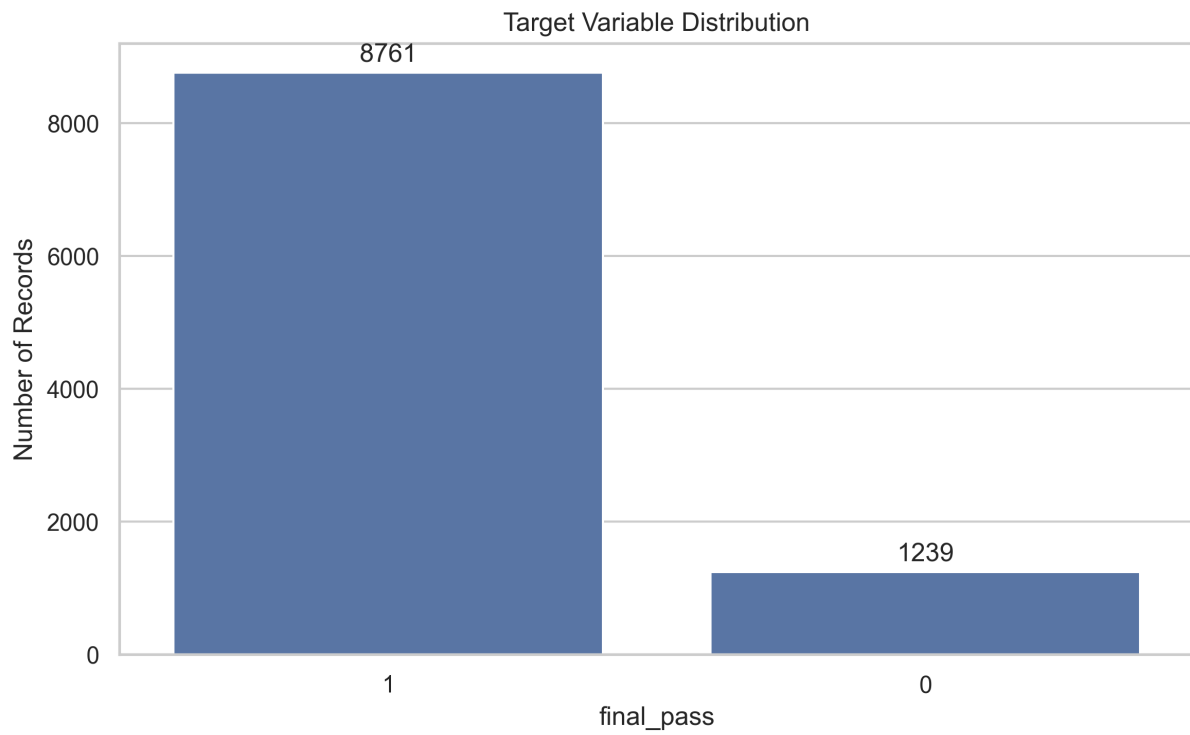


Figure 1. Class distribution of the target variable *final_pass* — 87.6% pass vs. 12.4% potentially problematic.

4. Exploratory Data Analysis (EDA)

4.1 What is EDA, and Why Do It?

Exploratory Data Analysis is the process of examining a dataset's structure, distributions, and relationships *before* building any model. The goal is to understand what the data actually looks like - are features skewed, correlated, or noisy? Does the target vary meaningfully across categories? Are there red flags like leakage or bad data? Skipping EDA and jumping straight to modeling risks building a model on flawed assumptions - EDA is where those assumptions get tested.

4.2 Univariate & Bivariate Analysis

Univariate analysis examines one variable at a time - its distribution, range, and shape (e.g. histograms of `cost_usd`, count plots of `defect_type`). This surfaces skewness, unusual spread, and rare categories early. **Bivariate analysis** examines the relationship between one variable and the target (e.g. pass rate by plant, pass rate by shift) - this is what reveals which operational factors actually correlate with quality outcomes, and it directly informed which features were worth engineering further.

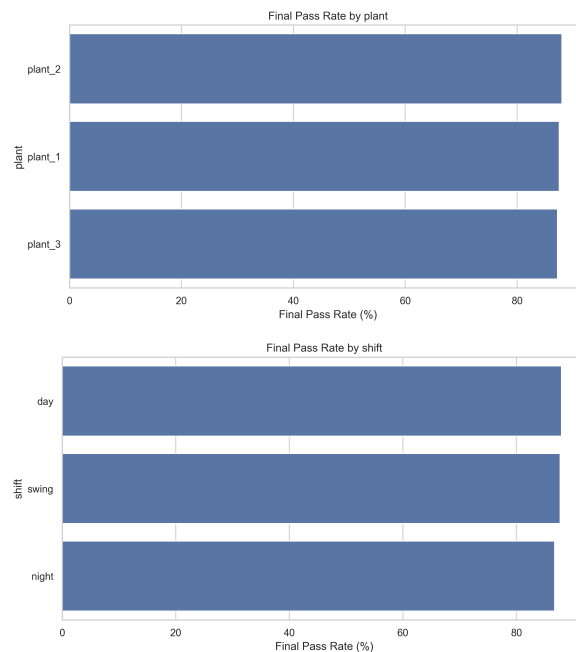


Figure 2. Pass rate by plant and by shift — bivariate analysis showing operational variation across the factory.

4.3 Correlation Analysis

A correlation heatmap measures how strongly each pair of numerical features moves together, on a scale from -1 to +1. It matters for two reasons: (1) two highly correlated features provide largely redundant information to the model, and (2) very high correlation with a post-outcome field can be a leakage warning sign. In this dataset, `rework_time_min` was found to correlate 0.92 with `total_cycle_time_min`, and `total_cycle_time_min` correlated 0.90 with `energy_kwh` - both flagged for review during feature selection.

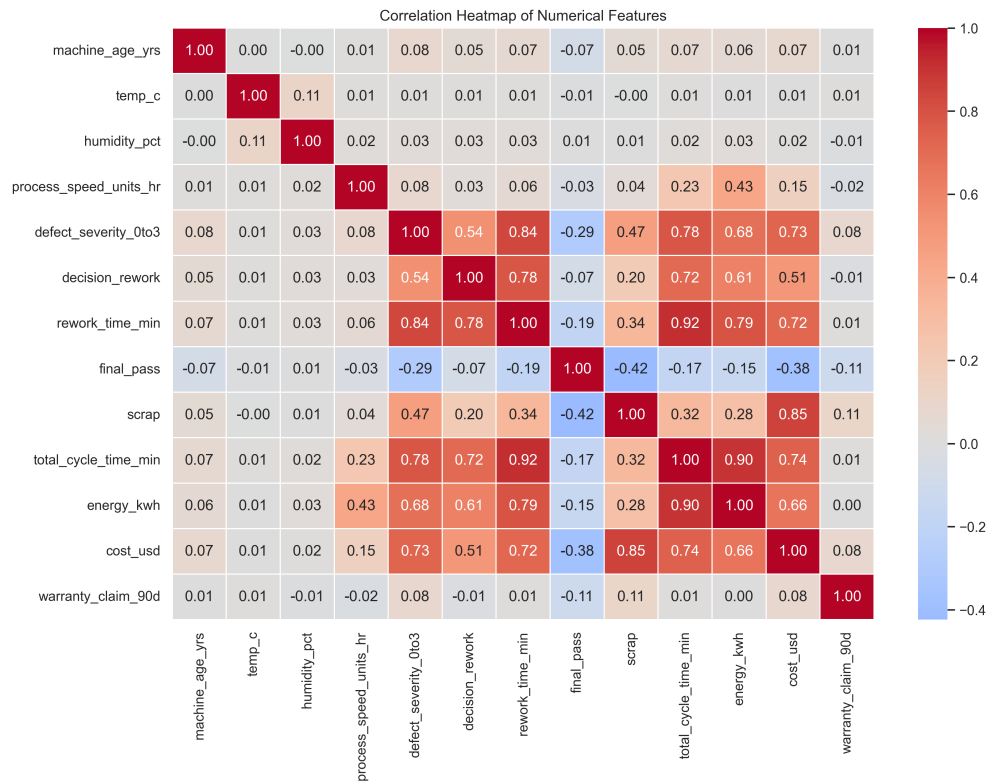


Figure 3. Correlation heatmap of numerical features.

4.4 Outliers & Skewness

Outliers are values far outside a variable's typical range - they can represent genuine rare events (e.g. an unusually long cycle time from a real production hiccup) or data-entry errors. **Skewness** measures how asymmetric a distribution is - many continuous manufacturing metrics (e.g. cost, cycle time) are naturally right-skewed, with a long tail of higher values. Rather than automatically deleting outliers or force-transforming every skewed feature, each case was investigated individually, since manufacturing outliers are often meaningful signal rather than noise - a very high-cost, long-cycle-time unit is exactly the kind of case the model should learn to be suspicious of.

4.5 Key EDA Findings (Summary)

- **Target imbalance:** ~87.6% pass vs. ~12.4% problematic — evaluation must go beyond accuracy.
- **Operational variation:** pass rates vary meaningfully by plant, line, shift, and material grade.
- **Strong correlations:** `rework_time_min` ↔ `total_cycle_time_min` (0.92); `total_cycle_time_min` ↔ `energy_kwh` (0.90).
- **Skew and outliers** present in several continuous features — investigated, not blindly removed.
- **Potential leakage features identified:** `decision_rework`, `rework_time_min`, `defect_severity_0to3`, `scrap`, `warranty_claim_90d` — flagged for a formal availability assessment before Feature Engineering.

5. Feature Engineering & Preprocessing

Feature engineering converts the EDA-validated raw dataset into a machine-learning-ready feature set. This phase is where most of the modeling project's real judgment calls happen - deciding what information the model is allowed to see, and in what form.

5.1 Target Leakage - the Most Important Concept in This Phase

What it is: target leakage happens when a feature used to train the model contains information that would not actually be available at the moment a real prediction is needed - often because that feature is itself a consequence of the outcome being predicted, or is recorded *after* it. A model trained on leaked features can show excellent test-set metrics while being useless in production, because the leaking feature won't exist yet when a real prediction is needed.

Why it mattered here: the dataset contains fields like `rework_time_min`, `scrap`, and `warranty_claim_90d`, which only get recorded *after* the `final_pass` decision has already been made (rework happens because a unit failed; scrap and warranty claims happen after shipment). Including these would let the model "cheat" - effectively peeking at the answer. The prediction point was explicitly defined as: manufacturing process complete, inspection/process data available, but before rework/scrap/warranty outcomes are known.

Outcome: `rework_time_min`, `scrap`, and `warranty_claim_90d` were excluded from the model's input features entirely. This is also why `decision_rework` (whether rework was decided, a process-time signal) was kept, but `rework_time_min` (how long rework took, only known after the fact) was not - a clear illustration of the pre/post prediction-point boundary.

5.2 Train-Test Split

What it is: the labeled dataset is split into a training set (used to fit the model) and a held-out test set (used only to evaluate it afterward, never seen during training). **Why:** evaluating a model on data it was trained on gives an overly optimistic, misleading picture of how it will perform on genuinely new data. **Why stratified:** a plain random split could, by chance, put a disproportionate share of the rare minority class into either the train or test set. A *stratified* split preserves the original 87.6% / 12.4% class ratio in both sets, which is especially important with an already-small minority class.

Outcome: 8,000 training records / 2,000 test records (80/20 stratified split), with the same ~87.6% / 12.4% class balance preserved in both.

5.3 Derived (Engineered) Features

Beyond the raw columns, several new features were derived to expose patterns that were not directly visible in the raw data:

Derived Feature	Built From	Why
<code>event_hour</code> , <code>event_day_of_week</code> , <code>event_month</code>	<code>event_ts</code> (timestamp)	A raw timestamp isn't directly usable by a model; decomposing it lets the model learn time-of-day, weekday, and seasonal patterns without leaking future dates.
<code>machine_age_group</code> (New / Moderate / Old)	<code>machine_age_yrs</code>	Groups a continuous variable into interpretable bands, which can help capture non-linear age effects and is easier to explain to a business stakeholder.

Derived Feature	Built From	Why
plant_shift	plant + shift	Captures plant-and-shift-specific operating patterns that neither field alone would represent (interaction feature).
material_shift	material_grade + shift	Same idea — certain material grades may behave differently depending on which shift processes them.

5.4 Encoding & Scaling

One-hot encoding (categorical features): models like Random Forest need numbers, not text, and categorical values like "plant_1"/"plant_2" have no natural order — so each category is converted into its own binary (0/1) column (e.g. plant_plant_1, plant_plant_2, plant_plant_3). This avoids implying a false numeric ranking between categories that a simple label-to-number mapping would create.

Numerical scaling: numerical features were scaled so they sit on comparable ranges, avoiding a feature like cost_usd (tens to hundreds) dominating a feature like defect_severity_0to3 (0-3) purely because of its raw magnitude, in models sensitive to feature scale.

Outcome: after encoding, the raw 23 features expanded to 115 model-ready features (mostly from one-hot encoding machine_id, which alone has 60 unique values).

5.5 The Preprocessing Pipeline (ColumnTransformer)

What it is: scikit-learn's ColumnTransformer bundles the numerical pipeline (scaling) and the categorical pipeline (one-hot encoding) into a single saved object, applied consistently to any new data. **Why it matters:** it guarantees that training data, test data, and future real-world prediction inputs are all transformed *identically* — the exact same encoding categories, the exact same scaling parameters — which prevents a whole class of subtle bugs where training-time and inference-time preprocessing silently drift apart.

Outcome: the fitted pipeline was saved as models/preprocessing_pipeline.pkl and reused both by the evaluation notebooks and, unmodified, by the deployed Streamlit application - a single source of truth for preprocessing.

Preprocessing workflow: Input Features → Separate Numerical / Categorical → Numerical Pipeline (scaling) → Categorical Pipeline (one-hot encoding) → Combined via ColumnTransformer → ML-Ready Features.

6. Model Development

6.1 Framing the ML Problem

This is a **supervised binary classification** problem: given labeled historical examples (features + known final_pass outcome), learn a function that predicts the outcome for new, unlabeled examples. "Supervised" means the training data includes the correct answer; "binary classification" means the output is one of exactly two classes.

6.2 Candidate Algorithms, Explained

Four algorithms were evaluated as a baseline sweep before narrowing down to the two strongest candidates for tuning:

Logistic Regression

A linear model that estimates the probability of the positive class using a weighted sum of the input features passed through a sigmoid function. Simple, fast, and highly interpretable (each feature gets one coefficient), but it can only capture linear relationships between features and the outcome — it struggles when the true pattern is more complex.

Decision Tree

Splits the data repeatedly on the feature/threshold that best separates the classes at each step (e.g. "is cost_usd > 78?"), forming a tree of yes/no questions ending in a prediction. Easy to visualize and interpret, but a single tree tends to overfit — it can memorize quirks of the training data rather than learning generalizable patterns.

Random Forest

An ensemble of many decision trees, each trained on a random subset of the data and a random subset of features (a technique called bagging). Each tree votes / gives a probability, and the forest averages them. Averaging many imperfect, decorrelated trees cancels out individual trees' overfitting, giving a model that is both more accurate and more stable than any single tree, while remaining reasonably interpretable through feature importance.

XGBoost (Extreme Gradient Boosting)

Also an ensemble of decision trees, but built *sequentially* rather than independently — each new tree is trained specifically to correct the errors of the trees built so far (gradient boosting), with built-in regularization to control overfitting. It is often extremely accurate on structured/tabular data and was included as a strong benchmark against Random Forest.

6.3 Baseline Comparison (Untuned Models)

Before any tuning, all four candidates were trained with default settings to get an honest, apples-to-apples starting comparison:

Model	Accuracy	Min. Precision	Min. Recall	Min. F1	ROC-AUC
Logistic Regression	72.3%	22.1%	48.8%	30.4%	67.4%
Decision Tree	81.8%	29.3%	33.1%	31.1%	60.9%

Model	Accuracy	Min. Precision	Min. Recall	Min. F1	ROC-AUC
Random Forest	89.4%	69.7%	25.0%	36.8%	69.8%
XGBoost	90.2%	93.2%	22.2%	35.8%	69.3%

Even at this early, untuned stage, Random Forest and XGBoost were clearly the strongest candidates on ROC-AUC and minority F1-score, which is why hyperparameter tuning effort was focused on those two rather than all four.

6.4 Cross-Validation

What it is: instead of judging a model on a single train/validation split (which could be lucky or unlucky), 5-fold stratified cross-validation splits the training data into 5 equal parts, trains on 4 and validates on the 1 held out, and repeats this 5 times so every part gets a turn as the validation set. The 5 resulting scores are averaged.

Why: this gives a far more reliable estimate of how a model (and a given set of hyperparameters) will generalize, and "stratified" again ensures each fold keeps the same ~87.6/12.4 class ratio.

Model	CV Minority F1 (mean)
Random Forest	0.3827
XGBoost	0.3551

6.5 Hyperparameter Tuning

What are hyperparameters? Settings chosen *before* training that control how a model learns (e.g. how many trees, how deep each tree can grow) — as opposed to parameters the model learns automatically from data.

RandomizedSearchCV vs. GridSearchCV: GridSearchCV exhaustively tries every combination of a hyperparameter grid, which becomes computationally expensive as the grid grows. RandomizedSearchCV instead samples a fixed number of random combinations from the given ranges — it explores the search space more efficiently and often finds a near-optimal combination in a fraction of the time. Here, 30 random configurations were tested per model, evaluated using 5-fold cross-validation.

Hyperparameter	Tuned Value	What It Controls
n_estimators	188	Number of trees in the forest — more trees generally means a more stable average, at the cost of compute time.
max_depth	12	Maximum depth each tree can grow to — limits how complex (and how prone to overfitting) any single tree can become.
max_features	sqrt	Number of features considered at each split — using only a random subset ($\sqrt{\text{total}}$) decorrelates the trees, which is central to why Random Forest works.
min_samples_split	5	Minimum samples required to split a node further — a higher value prevents overly specific, noise-fitting splits.
min_samples_leaf	2	Minimum samples allowed in a leaf node — smooths predictions by preventing leaves based on just 1 data point.
class_weight	balanced	Explained in Section 6.6 — reweights the minority class during training.

6.6 Handling Class Imbalance

With only 12.4% of records in the minority class, a model trained normally will naturally bias toward predicting the majority class, since that minimizes overall error even while ignoring the class that matters most for this business problem. **class_weight='balanced'** counteracts this by automatically increasing the training penalty for misclassifying minority-class examples, in inverse proportion to how rare that class is — effectively telling the model "getting a minority case wrong costs you more than getting a majority case wrong."

Alternatives considered / worth knowing for discussion: other common approaches include SMOTE (Synthetic Minority Over-sampling Technique, which generates synthetic minority examples), random oversampling of the minority class, or random undersampling of the majority class. `class_weight` was used here as a simpler, no-data-duplication approach built directly into scikit-learn's estimators; SMOTE and resampling strategies are noted as candidate future improvements (Section 11).

7. Model Evaluation & Metrics

Because of the class imbalance established in Section 3.3, evaluation deliberately goes well beyond plain accuracy. This section explains each metric used, in the order they build on each other.

7.1 Confusion Matrix

The foundation every other metric is built from. It's a 2x2 table comparing predicted class against actual class: True Positives (correctly predicted problematic), True Negatives (correctly predicted pass), False Positives (predicted problematic, was actually fine — a false alarm), and False Negatives (predicted pass, was actually problematic — a missed defect).

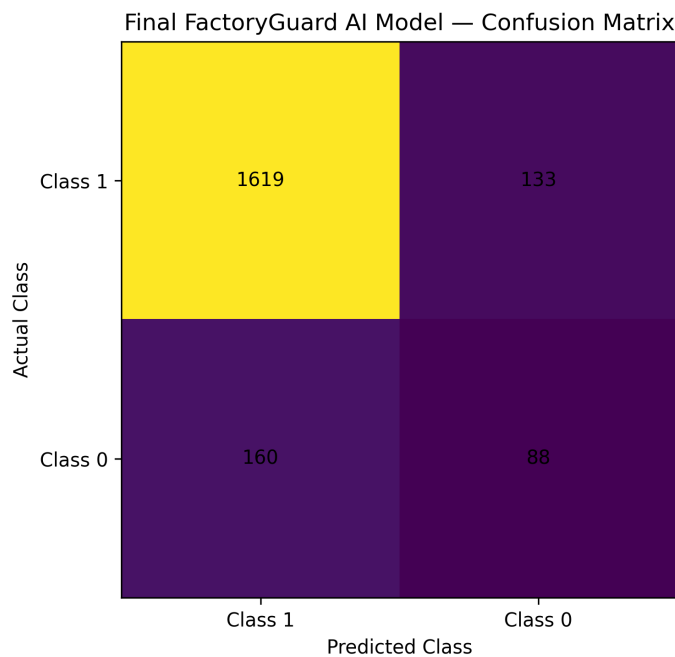


Figure 4. Confusion matrix of the final Random Forest model at threshold 0.50 (test set, $n=2,000$).

7.2 Precision, Recall, F1-score

Precision = True Positives / (True Positives + False Positives) — "of everything the model flagged as problematic, what fraction actually was?" High precision means few false alarms. **Recall** = True Positives / (True Positives + False Negatives) — "of everything that was actually problematic, what fraction did the model catch?" High recall means few missed defects. These two usually trade off against each other. **F1-score** is their harmonic mean — a single number that penalizes models which are lopsided (e.g. very high precision but very low recall), useful for comparing models with a single summary figure.

Why these matter more than accuracy here: in a business context, a false negative (missing a real defect) and a false positive (unnecessarily flagging a good unit) have very different costs — a missed defect can reach a customer, while a false alarm just costs extra inspection time. Precision and recall let you see and reason about this trade-off explicitly, which a single accuracy number hides completely.

7.3 ROC-AUC vs. PR-AUC

ROC-AUC (Receiver Operating Characteristic - Area Under Curve) plots True Positive Rate against False Positive Rate across all possible thresholds, and summarizes it into one number between 0.5 (no better than random guessing) and 1.0 (perfect separation). It measures how well the model ranks positive cases above negative cases, regardless of what threshold is finally chosen. **PR-AUC** (Precision-Recall - Area Under Curve) does the equivalent using precision and recall instead. On heavily imbalanced datasets, PR-AUC is generally considered more informative than ROC-AUC, because ROC-AUC can look deceptively good even when a model performs poorly on the rare class — that's why this project reports and tracks PR-AUC alongside ROC-AUC, not just ROC-AUC alone.

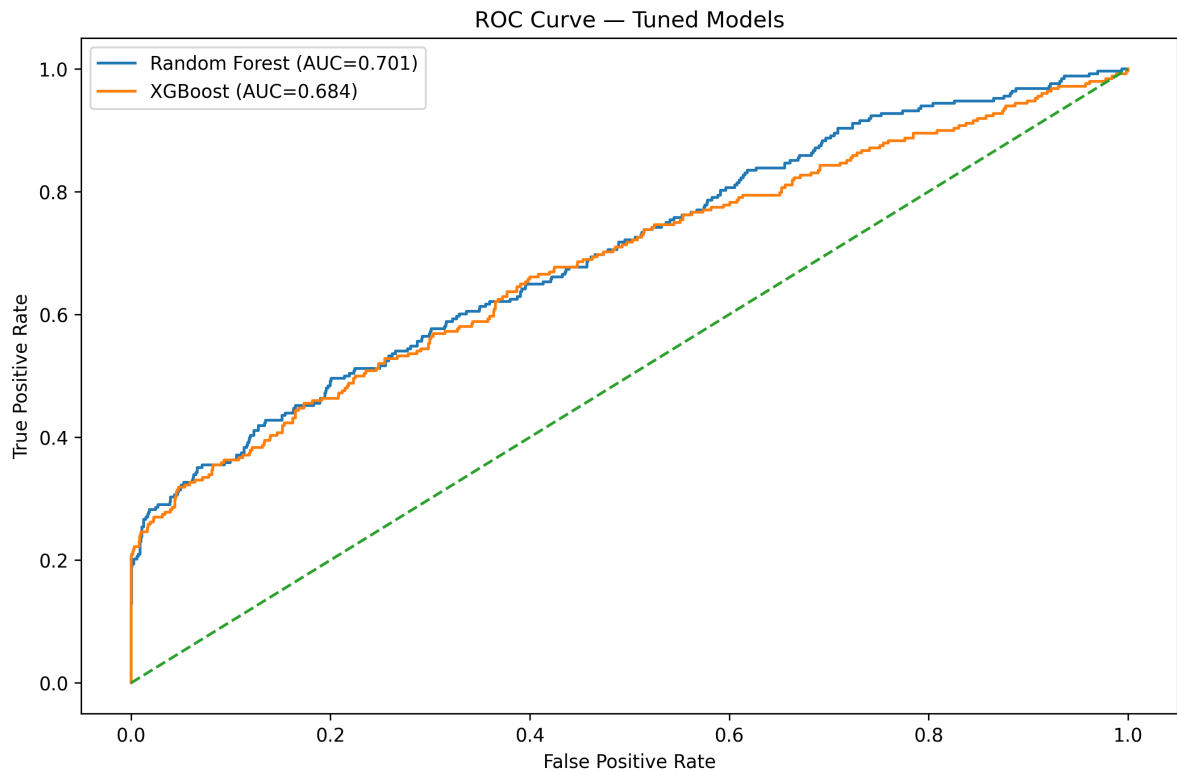


Figure 5. ROC curves comparing tuned Random Forest and XGBoost on the test set.

7.4 Final Model Selection: Random Forest vs. XGBoost

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Random Forest	85.35%	39.82%	35.48%	37.53%	70.09%	42.44%
XGBoost	89.70%	79.17%	22.98%	35.62%	68.39%	41.86%

This is the most important trade-off decision in the whole project, and a natural interview question: XGBoost achieved noticeably higher accuracy (89.7% vs. 85.35%) and much higher precision (79.2% vs. 39.8%) — on paper, it looks "better." But XGBoost's recall on the minority class was only 22.98%, compared to Random Forest's 35.48%. In plain terms: XGBoost is more accurate overall because it plays it safe and rarely flags anything as problematic (so it is very often right when it does), while Random Forest flags more units and, in doing so, catches meaningfully more of the real problem cases.

Since the entire point of FactoryGuard AI is to catch potentially problematic manufacturing units, missing them (false negatives) is the more costly error to minimize — so **Random Forest was selected as the final model** for its higher minority recall, F1-score, ROC-AUC, and PR-AUC, despite lower headline accuracy. This decision itself is a good example of why the "best" model in ML is defined by the business objective, not by whichever number happens to be highest.

7.5 Decision Threshold Tuning

What is a decision threshold? A classification model like Random Forest actually outputs a probability (e.g. "32% likely to be problematic"), not a hard label. The threshold is the cutoff probability above which a case is labeled "problematic." The default is 0.50, but it doesn't have to be — raising it makes the model more conservative (fewer flags, higher precision, lower recall); lowering it makes it more aggressive (more flags, lower precision, higher recall).

Metric	Threshold 0.50	Threshold 0.55
Accuracy	85.35%	87.35%
Minority Precision	39.82%	48.41%
Minority Recall	35.48%	30.65%
Minority F1	37.53%	37.53%
False Positive Rate	7.59%	4.62%
False Negative Rate	64.52%	69.35%

0.55 was tested as an alternative because it modestly improved cross-validated F1 (0.3858 vs. 0.3827). But evaluated on the held-out test set, it traded recall for precision without improving overall F1 — and since minority recall (catching real problems) is the priority here, the **default threshold of 0.50 was retained** as the final operating threshold.

8. Model Interpretation & Explainability

8.1 Feature Importance

What it is: Random Forest's built-in feature importance (Gini importance / mean decrease in impurity) measures, on average across all trees in the forest, how much each feature contributes to correctly splitting the data at the points it's used. A higher score means the feature was more consistently useful for separating the two classes.

Why it matters: it turns the model from a black box into something a business stakeholder can reason about — "the model relies most heavily on cost and defect severity" is a statement anyone in the business can sanity-check against their own intuition.

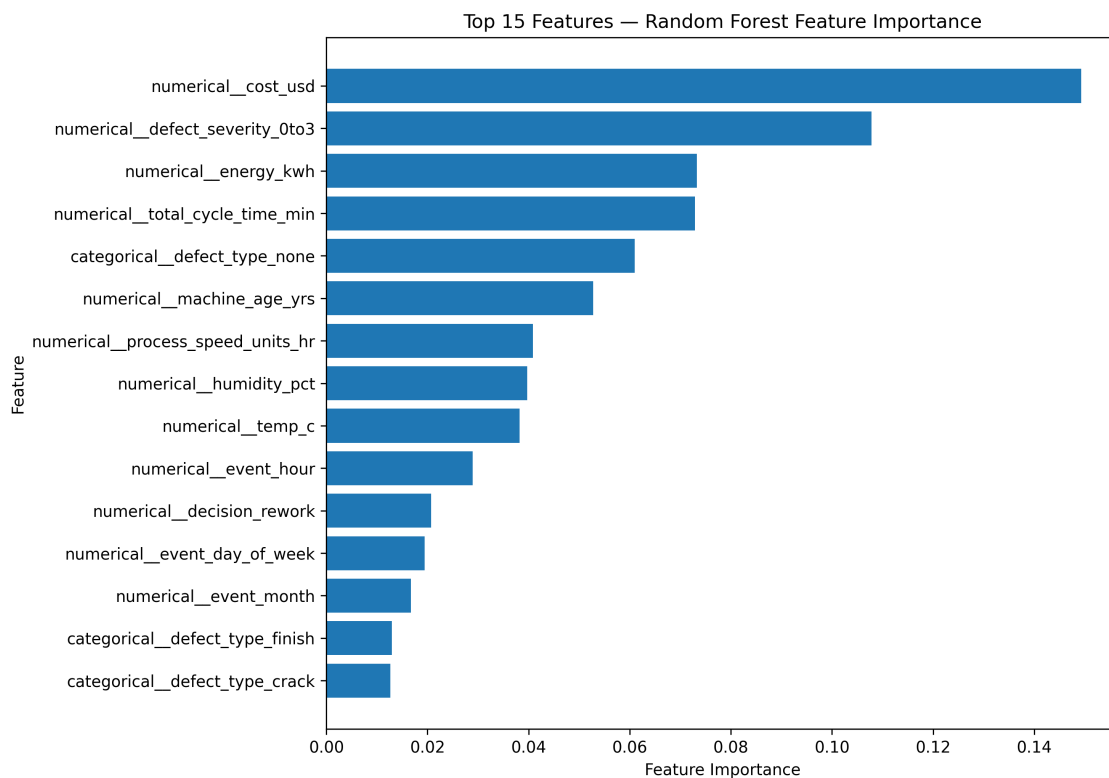


Figure 6. Top 15 features by Random Forest feature importance.

Rank	Feature	Importance
1	cost_usd	14.93%
2	defect_severity_0to3	10.78%
3	energy_kwh	7.33%
4	total_cycle_time_min	7.29%
5	defect_type = none	6.10%

8.2 Business Interpretation of the Results

- **Overall accuracy:** the model correctly classifies approximately 85% of test observations.
- **Minority-class detection:** the model catches approximately 35% of actual problematic cases.

- **Minority precision:** approximately 40% of units flagged as problematic are genuinely problematic.
- **ROC-AUC ≈ 0.70 :** meaningful, above-random ability to distinguish the two classes.
- **False positives (7.6% of passing units):** some good units are flagged, causing some unnecessary inspection.
- **False negatives (64.5% of problematic units):** a substantial share of real problems are still missed.
- **Recommended role:** a decision-support and screening system, not a fully automated quality gate.

9. Deployment & the Streamlit Application

9.1 Model Persistence (Why Save a Model At All?)

Training a Random Forest with 188 trees on 8,000 records takes real computation time. Re-training from scratch every time a prediction is needed would be slow and wasteful, and would also mean the model behind every prediction could silently differ run to run. **Joblib** serializes the trained model and the fitted preprocessing pipeline into .pkl files on disk once, after training - these are then simply loaded (not retrained) whenever a prediction is needed, guaranteeing every prediction uses the exact same, already-validated model.

9.2 Application Architecture

FactoryGuard AI — Deployment & Inference Architecture

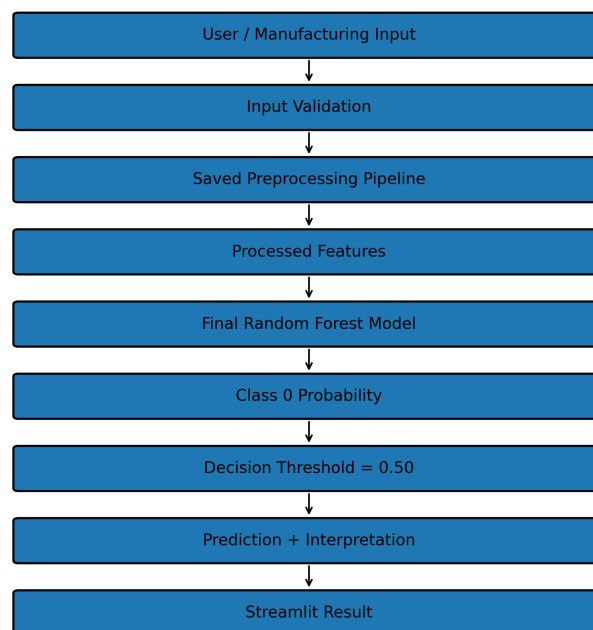


Figure 7. FactoryGuard AI deployment / inference architecture.

Inference pipeline: Manufacturing Input → Validation (src/validation.py) → Feature Engineering (src/feature_engineering.py) → Saved Preprocessing Pipeline → Final Random Forest Model → Class Probabilities → Decision Threshold (0.50) → Final Classification. Both the notebooks and the deployed Streamlit app call these same src/ modules, so there is one single, tested source of truth for the pipeline logic rather than duplicated code that could drift apart.

Input validation, specifically: before any prediction runs, incoming input is checked for the correct columns, no missing values, numeric values within the ranges actually observed in training data, and categorical values that match a category the model has actually seen before. This exists because a model given an out-of-distribution input (e.g. a plant name it's never seen) will still silently produce *some* prediction - validation catches that before it happens rather than trusting an unreliable output.

9.3 Why Streamlit, and the Application Pages

Why Streamlit: it turns a Python script into an interactive web app with minimal boilerplate - no separate frontend framework or JavaScript needed - which made it a fast way to put the trained model in front of a non-technical user.

Page	Purpose
Home	Project overview, model snapshot, dataset summary, architecture diagram
Predict	Score a single observation via a guided form, with one-click example scenarios
What-If Explorer	Live, slider-driven exploration plus a single-feature sensitivity chart
Model Insights	Performance metrics, feature importance, documented strengths/limitations
Batch Prediction	Upload a CSV of many observations, score them all at once, download results

GitHub Repository:

<https://github.com/Bhardwaj-Arin/FactoryGuard-AI-Intelligent-Manufacturing-Quality-Decision-System>

Live Streamlit App: <https://factoryguard-ai-intelligent-manufacturing-quality-decision-sys.streamlit.app/>

10. Technology Stack

Layer	Technology	Why It Was Used
Language	Python	Standard ML language; huge ecosystem and readability.
Data processing	Pandas, NumPy	Structured data handling, cleaning, aggregation; NumPy powers the numerical operations underneath.
Visualization	Matplotlib, Seaborn	Matplotlib for core plotting; Seaborn for higher-level statistical charts (heatmaps, boxplots) with less code.
Machine learning	scikit-learn, XGBoost	scikit-learn for preprocessing pipelines, Random Forest, cross-validation, and metrics; XGBoost as a high-performance gradient boosting benchmark.
Explainability	SHAP (available in stack)	Supports deeper, per-prediction explanations beyond built-in feature importance.
Model persistence	Joblib	Fast, reliable serialization of the trained model and preprocessing pipeline.
Application	Streamlit	Turns the trained pipeline into an interactive web app with minimal code.
Dev environment	VS Code, Jupyter Notebook	Notebooks for exploration/experimentation; VS Code + src/ for reusable, production-style code.
Version control	Git & GitHub	History, collaboration, and public/portfolio hosting of the project.
Deployment	Streamlit Community Cloud	Free, straightforward hosting for a Streamlit app directly from GitHub.

11. Limitations & Future Improvements

Documented Limitations

Limitation	Evidence
Low minority-class recall	Minority recall = 35.48%
High false-negative rate	False Negative Rate = 64.52%
Moderate minority F1-score	Minority F1 = 37.53%
Class imbalance	Class 0 represents approximately 12.39% of observations
Dataset dependence	Performance depends on the quality/representativeness of available data
Feature dependence	Predictions depend on features available during training and inference
Potential distribution shift	Real-world manufacturing conditions may differ from training data
No causal interpretation	Feature importance and associations do not establish causation

Potential Future Improvements

Area	Potential Improvement
Data collection	Collect more representative minority-class observations
Class imbalance	Evaluate SMOTE, other resampling techniques, or cost-sensitive learning
Threshold optimization	Optimize the threshold against the actual business cost of false negatives/positives
Modeling	Evaluate additional models and ensemble strategies
Explainability	Introduce more detailed local (per-prediction) explanations, e.g. SHAP
Monitoring	Monitor model performance and data distribution after deployment

12. Conclusion

FactoryGuard AI demonstrates a complete, well-documented applied machine learning workflow - from business problem framing through leakage-aware feature engineering, comparative model development, threshold analysis, interpretability, and deployment as a live, interactive web application.

The final Random Forest model achieves meaningful class separation ($\text{ROC-AUC} \approx 0.70$) and strong overall accuracy ($\approx 85\%$), while candidly documenting its principal limitation: recall on the minority "problematic" class remains moderate ($\approx 35\%$). Every major decision in the pipeline - from excluding leakage features, to choosing Random Forest over a higher-accuracy XGBoost, to keeping the default 0.50 threshold - was driven by the same underlying business priority: catching real quality problems matters more than a high headline accuracy number.

The project's modular structure - shared src/ modules used identically by both the notebooks and the Streamlit application - and its thorough phase-by-phase documentation position it well for future iteration.